

Competitive Programming

Section 55: Randomization And Hashing

Competitive Programming Course

2026-07-10

Table of contents

1	Hashing discipline	3
1.1	Polynomial rolling hash	3
1.2	Palindrome hash	3
1.3	Anagram substring queries	3
1.4	Collision policy	4
1.5	Substring hash indexing example	4
1.6	Exact alternative for anagrams	4
1.7	Double-hash checklist	5
1.8	Testing hash code	5
1.9	When not to hash	5
2	Exercises	6
3	Query contracts and exact oracles	7

1 Hashing discipline

Hashing is a speed tool, not a proof replacement. Know the collision risk, use deterministic checks when constraints allow, and reserve randomized hashes for situations where exact comparison is too expensive.

1.1 Polynomial rolling hash

For string s , choose base B and modulus M :

```
H[i+1] = H[i] * B + value(s[i])
```

Then substring hash $[l, r)$ is:

```
H[r] - H[l] * B^(r-l)
```

Normalize modulo M .

To compare substrings, compare their normalized hashes. To reduce collision risk, use two moduli or a 64-bit unsigned hash.

1.2 Palindrome hash

Build hashes for the string and the reversed string. Substring $s[l..r]$ is a palindrome iff it equals its reverse:

```
hash_s(l, r) == hash_rev(n-1-r, n-1-l)
```

Use half-open intervals carefully. Most bugs in hash problems are index conversion bugs, not hash math bugs.

1.3 Anagram substring queries

For lowercase letters, hashing is unnecessary. Maintain 26 prefix counts:

```
cnt[c][i+1] = cnt[c][i] + (s[i] == c)
```

Two substrings are anagrams iff all 26 character counts match. This is exact, fast, and deterministic.

1.4 Collision policy

In real contests:

- If exact comparison is cheap enough, do it.
- If hashing is needed, state that equality of hashes is treated as equality with negligible collision probability.
- For adversarial platforms, prefer double hashing or deterministic algorithms.
- Randomize the base when hacks are possible.

Hashing should make an already-correct model fast; it should not be the only reason you believe a solution is correct.

1.5 Substring hash indexing example

Use half-open intervals $[l, r)$.

String:

```
s = abac
```

If:

```
H[i+1] = H[i] * B + value(s[i])
```

then the hash of $s[1:3] = \text{"ba"}$ is:

```
H[3] - H[1] * B^2
```

The exponent is the substring length $r-l$, not r and not l .

1.6 Exact alternative for anagrams

For anagram substring queries, prefix frequency vectors are better than hashes:

```
freq[c][r] - freq[c][l]
```

This gives the exact count of character c in $[l, r)$. Two substrings are anagrams iff all counts match. There is no collision risk.

1.7 Double-hash checklist

If you use hashing for adversarial substring equality:

```
choose two large prime moduli
precompute powers for both
store two prefix arrays
compare pairs of hashes
```

A collision would need to collide under both moduli. This still is not a mathematical proof of equality, but it is standard in contests when exact suffix structures are too expensive to implement.

When the alphabet is small and the query is about multisets, prefer prefix counts. They are exact and often simpler.

1.8 Testing hash code

Hash bugs usually come from indexing, so test substrings that touch boundaries:

```
whole string
first character
last character
two equal substrings at different positions
two almost-equal substrings
```

For palindrome hashes, test odd and even lengths separately. Also test a single-character substring; it should always be a palindrome.

1.9 When not to hash

Do not hash if an exact solution is simple enough:

```
small substring length -> direct comparison
small alphabet anagram -> prefix counts
palindrome around centers -> Manacher
ordered suffix questions -> suffix array/tree/automaton
```

Hashing is excellent for speed, but exact structure is better when the problem requires a proof with no collision caveat.

2 Exercises

- A. Equal Substrings: Answer whether two same-length substrings are equal.
- B. Palindrome Substrings: For each query, say whether the first substring is a palindrome.
- C. Anagram Substrings: Answer whether two same-length substrings have the same multiset of letters. # Contest checklist

3 Query contracts and exact oracles

For a polynomial prefix hash, store $H[i+1] = H[i] * B + \text{code}(s[i])$. Normalize a substring $[l, r)$ by subtracting $H[l] * B^{(r-l)}$. Equal normalized hashes are a fast probabilistic equality test; use two moduli or an exact comparison when a collision is unacceptable.

For palindrome $[l, r)$, compare its forward hash with the corresponding range in the reversed string. Write the reverse-index conversion before coding: $[l, r)$ maps to $[n-r, n-l)$. On `abacaba`, $[2, 5)$ is `aca` and maps to the same reversed interval.

Anagrams need no randomness locally: 26 prefix counts make each character count in $O(1)$, and two substrings are anagrams iff all 26 differences match. This is exact and preferable to randomized character weights for lowercase input.

In C++ use `long long` for products before modulo; in Python use integers and avoid slicing in the optimized query loop. Test equal ranges, equal hashes at different positions, length one, full string, and reverse-index boundaries. Run `CP_TARGET=solution python3 sections/55_randomization_and_hashing/check.py` and compare random short queries with direct slices/sorted character counts.

State the invariant, choose the model, prove the operation, then code the smallest reliable version.